

A Reproducible Benchmarking Framework for CPU and NVIDIA CUDA GPU Machine Learning Performance Analysis with NVIDIA Nsight Systems

William Lorenzo^{1*}

^{1*}Department of Statistics, B.S.: Data Science Concentration,
California State University, East Bay, 25800 Carlos Bee Boulevard,
Hayward, California, 94542, USA.

Corresponding author(s). E-mail(s): wlorenzo@horizon.csueastbay.edu;

Abstract

Graphics processing units have become central to modern machine learning because they accelerate highly parallel numerical workloads through thousands of lightweight execution threads. This study presents GPU-MLBench, a reproducible benchmarking workflow for comparing CPU and NVIDIA GPU implementations of common machine learning algorithms. The Forest Cover Type dataset was used as a multi-class classification benchmark containing 581,012 observations, 54 predictor variables, and seven cover-type classes. Four models were evaluated: Random Forest, XGBoost, TensorFlow neural networks, and PyTorch neural networks. CPU and GPU implementations were compared using training time, inference time, classification accuracy, peak GPU memory usage, average GPU utilization, and CPU-to-GPU speedup. Results showed that GPU acceleration was highly workload dependent. XGBoost achieved the strongest acceleration, reducing training time from 526.56 seconds on CPU to approximately 19.56 seconds on GPU, corresponding to a $26.92\times$ measured speedup. Random Forest achieved a moderate $2.83\times$ speedup, PyTorch achieved a small $1.17\times$ speedup, and TensorFlow GPU execution was slower than CPU execution for the selected multilayer perceptron configuration. NVIDIA Nsight Systems profiling was used to inspect CPU utilization, host-side queueing, and thread activity during GPU-accelerated execution. Overall, the study demonstrates that NVIDIA GPU acceleration can substantially improve machine learning throughput, but the magnitude of improvement depends on algorithm structure,

workload size, framework implementation, data movement overhead, and GPU utilization efficiency.

Keywords: NVIDIA GPU acceleration, CUDA, RAPIDS cuML, XGBoost, TensorFlow, PyTorch, Nsight Systems, machine learning benchmarking, GPU profiling, Forest Cover Type dataset

1 Introduction

Graphics processing units are widely used in machine learning because many training algorithms contain matrix operations, gradient computations, histogram construction, and ensemble operations that can be parallelized. NVIDIA CUDA provides a programming and execution model that allows machine learning libraries to use the GPU as a numerical accelerator. However, GPU acceleration is not automatically faster than CPU execution. Performance depends on the size of the workload, the amount of data movement between host and device memory, kernel launch overhead, model complexity, and the degree to which the algorithm exposes parallelism.

The purpose of this study was to evaluate how much performance improvement an NVIDIA GPU can provide across different machine learning workloads. The project compares CPU and GPU implementations of Random Forest, XGBoost, TensorFlow neural networks, and PyTorch neural networks using a consistent dataset and benchmark logging pipeline. The central research question is:

How much performance improvement can NVIDIA GPU acceleration provide for selected machine learning algorithms compared with CPU execution?

This paper is organized as a reproducible computational research study. It describes the dataset, hardware and software environment, GPU-MLBench system architecture, mathematical performance metrics, model-specific benchmark design, exploratory figures, profiling results, limitations, and full Python source code.

2 Dataset Source and Study Design

The Forest Cover Type dataset was used as the benchmark dataset. Each observation represents a forest region described by cartographic and environmental variables. The prediction task is a seven-class classification problem in which the target variable is forest cover type.

Let the dataset be represented as

$$D = \{(x_i, y_i)\}_{i=1}^n, \quad (1)$$

where $x_i \in \mathbb{R}^{54}$ is the feature vector for observation i , $y_i \in \{0, 1, \dots, 6\}$ is the encoded class label, and $n = 581,012$ is the number of observations.

The dataset was divided into training and testing subsets:

$$D = D_{\text{train}} \cup D_{\text{test}}, \quad D_{\text{train}} \cap D_{\text{test}} = \emptyset. \quad (2)$$

The training set contained 80% of the observations, and the testing set contained 20% of the observations. For neural network models, continuous features were standardized according to

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}, \quad (3)$$

where x_{ij} is feature j for observation i , μ_j is the training-set mean of feature j , and σ_j is the training-set standard deviation.

3 NVIDIA GPU Computing Background

A CPU contains a relatively small number of powerful cores optimized for sequential execution, branching, and general-purpose system tasks. A GPU contains many parallel execution units designed for high-throughput numerical computation. NVIDIA GPUs use a Single Instruction Multiple Thread execution model in which groups of threads execute the same instruction over different data elements.

Let N_c denote the number of available CUDA cores and let f denote the operating clock frequency. A simplified expression for potential arithmetic throughput is

$$P \propto N_c f. \quad (4)$$

This expression is not a complete hardware performance model because actual throughput also depends on memory bandwidth, instruction mix, occupancy, cache behavior, and kernel efficiency. Nevertheless, it highlights why algorithms with large amounts of data-parallel work can benefit from GPU acceleration.

The total GPU execution time can be decomposed as

$$T_{\text{GPU, total}} = T_{\text{transfer}} + T_{\text{kernel}} + T_{\text{sync}} + T_{\text{overhead}}, \quad (5)$$

where T_{transfer} is host-to-device or device-to-host data movement time, T_{kernel} is GPU kernel execution time, T_{sync} is synchronization time, and T_{overhead} includes framework and scheduling overhead.

4 Hardware and Software Environment

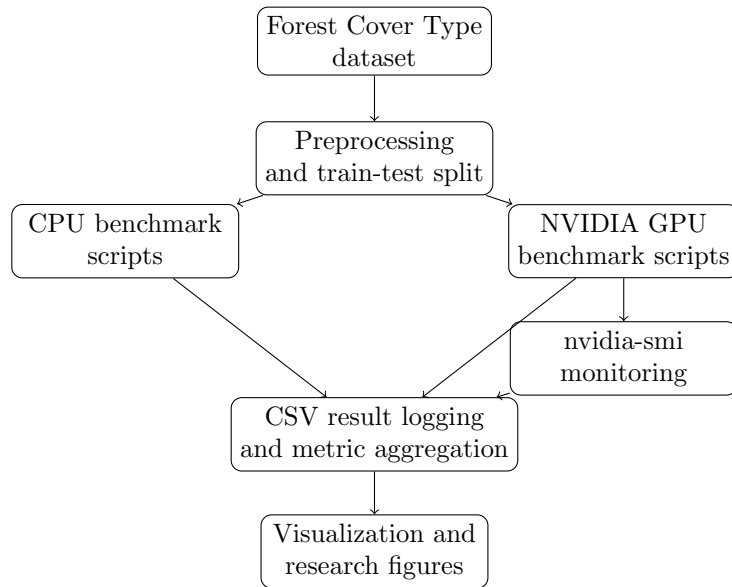
The benchmark was executed on a workstation using an NVIDIA GeForce RTX 3060 GPU. Table 1 summarizes the experimental environment.

5 GPU-MLBench Framework Architecture

GPU-MLBench was organized as a modular Python benchmarking framework. The dataset module loaded and preprocessed the Forest Cover Type data. Model-specific scripts trained CPU and GPU versions of each algorithm. A shared metrics module logged timing, accuracy, library versions, and model parameters. A GPU monitoring module sampled `nvidia-smi` to estimate GPU memory consumption and utilization. Optional NVTX ranges were used to annotate workloads for NVIDIA Nsight Systems profiling.

Table 1 Experimental hardware and software environment.

Component	Specification
GPU	NVIDIA GeForce RTX 3060, 12 GB VRAM
CPU	12th Gen Intel processor environment
Operating system	Ubuntu 22.04 on WSL2
CUDA support	NVIDIA CUDA-enabled execution
RAPIDS cuML	GPU Random Forest implementation
XGBoost	CPU and CUDA histogram implementations
TensorFlow	CPU and GPU neural network training
PyTorch	CPU and CUDA neural network training
Profiler	NVIDIA Nsight Systems
Dataset	Forest Cover Type


Fig. 1 GPU-MLBench architecture showing dataset ingestion, preprocessing, CPU baselines, NVIDIA GPU implementations, GPU monitoring, result logging, and visualization generation.

6 Experimental Methodology

6.1 Benchmark timing

Training time was measured using Python’s high-resolution performance counter. If t_s is the start timestamp and t_e is the end timestamp, then training time is

$$T = t_e - t_s. \quad (6)$$

For GPU workloads, synchronization was performed when supported so that asynchronous CUDA operations were completed before timing ended.

6.2 Speedup ratio

The CPU-to-GPU speedup ratio was computed as

$$S = \frac{T_{\text{CPU}}}{T_{\text{GPU}}}. \quad (7)$$

A value $S > 1$ indicates that GPU execution was faster than CPU execution, while $S < 1$ indicates that GPU execution was slower.

6.3 Classification accuracy

For a test set of m observations, classification accuracy was computed as

$$\text{Accuracy} = \frac{1}{m} \sum_{i=1}^m \mathbb{I}(\hat{y}_i = y_i), \quad (8)$$

where $\hat{y}_i$ is the predicted class, y_i is the true class, and $\mathbb{I}(\cdot)$ is the indicator function.

6.4 GPU utilization and memory

Average GPU utilization was estimated from k sampled values:

$$U_{\text{avg}} = \frac{1}{k} \sum_{j=1}^k U_j, \quad (9)$$

where U_j is the sampled utilization percentage at time step j . Peak GPU memory usage was computed as

$$M_{\text{peak}} = \max(M_1, M_2, \dots, M_k), \quad (10)$$

where M_j is the sampled GPU memory usage in megabytes.

7 Machine Learning Benchmarking

7.1 Random Forest

The CPU Random Forest benchmark used scikit-learn, while the GPU Random Forest benchmark used RAPIDS cuML. Random Forest constructs an ensemble of decision trees and predicts the majority class:

$$\hat{f}(x) = \text{mode}\{h_1(x), h_2(x), \dots, h_B(x)\}, \quad (11)$$

where $h_b(x)$ is the prediction of tree b and B is the number of trees.

7.2 XGBoost

XGBoost builds an additive ensemble of regression trees. The prediction for observation i is

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), \quad (12)$$

where f_k is tree k . The regularized objective is

$$\mathcal{L} = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k), \quad (13)$$

where l is the loss function and Ω penalizes model complexity. The CPU benchmark used histogram tree construction on the CPU, while the GPU benchmark used CUDA acceleration.

7.3 TensorFlow and PyTorch neural networks

The TensorFlow and PyTorch neural networks used a multilayer perceptron architecture:

$$54 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 7. \quad (14)$$

For hidden layer l , the forward transformation is

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)}), \quad (15)$$

where $W^{(l)}$ is the weight matrix, $b^{(l)}$ is the bias vector, and σ is the ReLU activation function:

$$\sigma(z) = \max(0, z). \quad (16)$$

The output layer used softmax probabilities:

$$P(y = c \mid x) = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}}, \quad (17)$$

where $C = 7$. The training objective was cross-entropy loss:

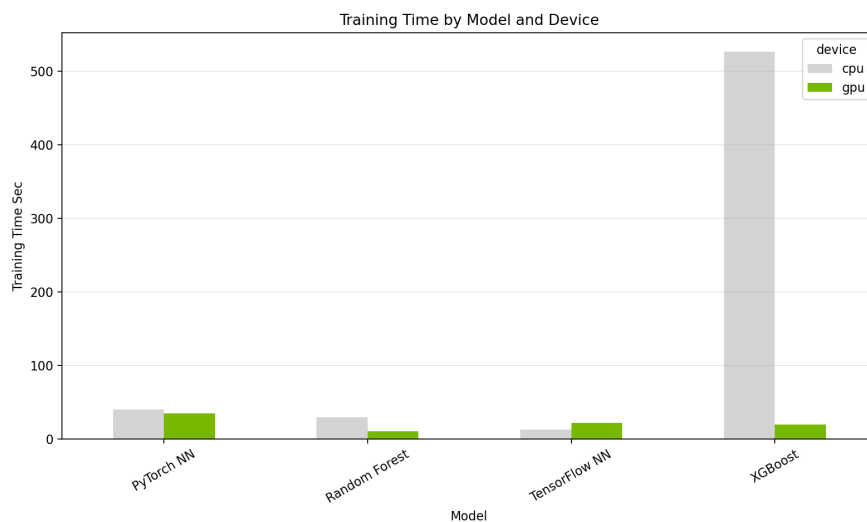
$$\mathcal{L} = - \sum_{c=1}^C y_c \log(\hat{p}_c). \quad (18)$$

8 Results

Table 2 summarizes the benchmark results. XGBoost achieved the largest acceleration, while TensorFlow GPU execution was slower than CPU execution for the tested model architecture.

Table 2 CPU and NVIDIA GPU benchmark results.

Model	CPU time (s)	GPU time (s)	Speedup	GPU accuracy	Peak memory (MB)
Random Forest	29.90	10.56	2.83×	0.7982	1684
XGBoost	526.56	19.56	26.92×	0.9199	1272
TensorFlow NN	12.80	21.58	0.59×	0.8325	1514
PyTorch NN	40.36	34.60	1.17×	0.8189	1076


Fig. 2 Training time comparison between CPU and GPU implementations. XGBoost showed the largest reduction in training time when GPU acceleration was used.

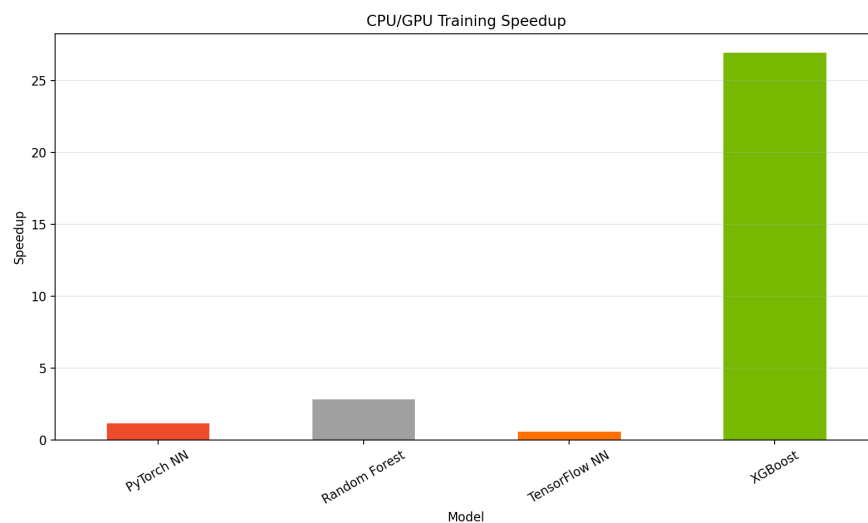


Fig. 3 CPU-to-GPU speedup by model. XGBoost achieved the largest acceleration, while Tensor-Flow GPU execution was slower than CPU execution for the selected neural network configuration.

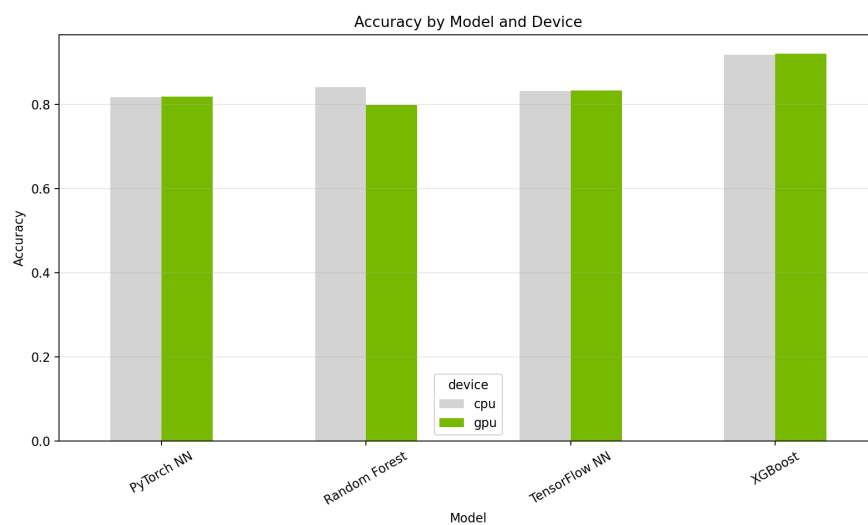


Fig. 4 Accuracy comparison by model and device. Accuracy remained generally comparable across CPU and GPU implementations.

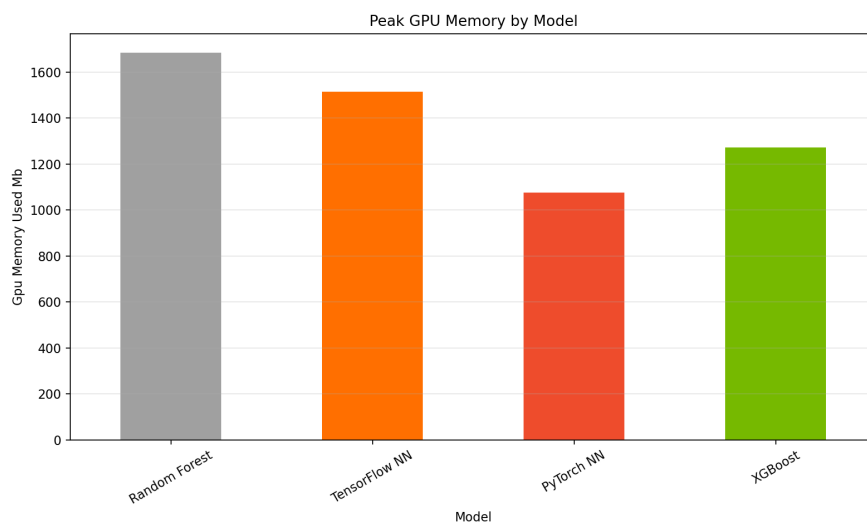


Fig. 5 Peak GPU memory usage by model. Random Forest used the largest measured GPU memory, followed by TensorFlow, XGBoost, and PyTorch.

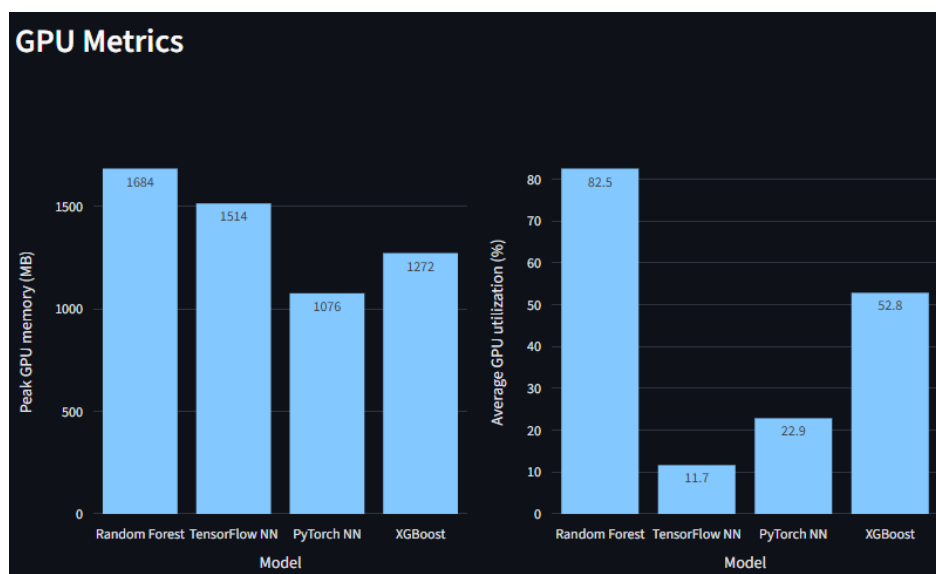


Fig. 6 GPU metrics dashboard showing peak memory usage and average GPU utilization across benchmarked models.

9 NVIDIA Nsight Systems Profiling

NVIDIA Nsight Systems was used to inspect CPU and thread-level behavior during GPU-accelerated execution. The profiler captured CPU utilization, host-side queueing, synchronization behavior, and thread scheduling. These traces are useful because GPU-accelerated workloads still require CPU coordination for data preparation, kernel launches, framework scheduling, and result collection.

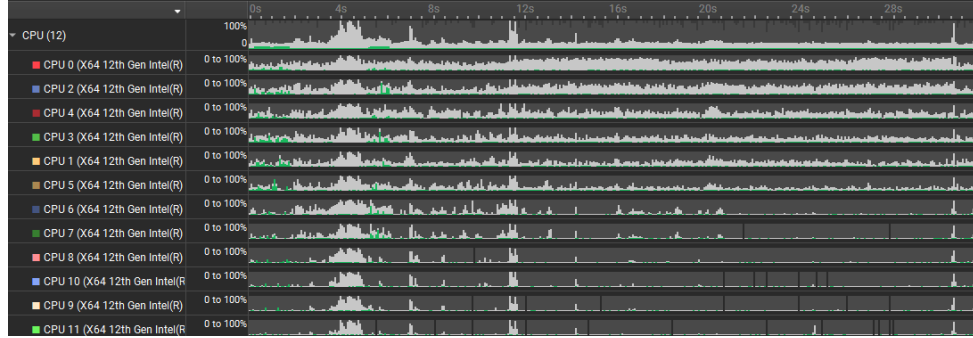


Fig. 7 CPU utilization during benchmark execution. The CPU remained active during scheduling, coordination, and host-side workload management.

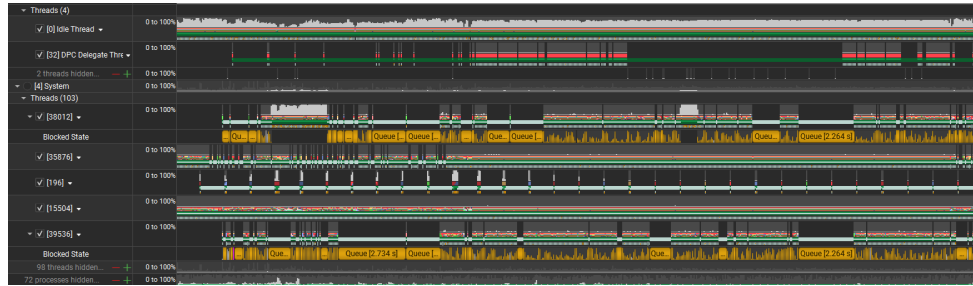


Fig. 8 Nsight Systems thread activity showing queueing, blocked states, synchronization, and CPU-side coordination during GPU-accelerated execution.

10 Discussion

The results demonstrate that GPU acceleration is strongly workload dependent. XGBoost produced the largest speedup because its histogram-based tree construction can exploit parallel computation efficiently on the GPU. The GPU implementation reduced training time from more than 500 seconds to less than 20 seconds while maintaining the highest accuracy in the benchmark.

Random Forest achieved a moderate speedup. This indicates that ensemble tree methods can benefit from GPU acceleration, but the magnitude of improvement may be limited by implementation details, memory access patterns, and host-device data

transfer. PyTorch achieved only a small speedup, suggesting that the tested neural network was not large enough to fully utilize the GPU. TensorFlow was slower on the GPU than on the CPU for this configuration, which suggests that GPU launch overhead, input pipeline behavior, or model size may have outweighed the benefit of parallel execution.

The accuracy results show that GPU acceleration did not materially degrade predictive performance. Accuracy differences between CPU and GPU implementations were small for most models. The main benefit of GPU acceleration was therefore computational throughput rather than improved predictive quality.

11 Limitations

Several limitations should be acknowledged. First, the benchmark used a single NVIDIA RTX 3060 GPU, so results may not generalize to higher-end GPUs, data center GPUs, or multi-GPU systems. Second, only one dataset was used. Third, the neural network architecture was intentionally simple, which may have limited the ability of TensorFlow and PyTorch to benefit from GPU execution. Fourth, the environment used WSL2, which may introduce overhead not present in native Linux installations. Fifth, the benchmarks were based on a limited number of runs, so future work should include repeated trials, confidence intervals, and statistical significance testing. Finally, this study did not evaluate distributed training, mixed precision tuning, or advanced GPU memory optimization strategies.

12 Conclusion

This study presented GPU-MLBench, a reproducible benchmark workflow for comparing CPU and NVIDIA GPU machine learning workloads. Using the Forest Cover Type dataset, four models were evaluated across CPU and GPU implementations: Random Forest, XGBoost, TensorFlow neural networks, and PyTorch neural networks. The results showed that XGBoost benefited most from GPU acceleration, achieving a measured $26.92\times$ speedup while maintaining strong accuracy. Random Forest achieved moderate acceleration, PyTorch achieved a small acceleration, and TensorFlow was slower on GPU for the tested model. NVIDIA Nsight Systems profiling further showed that CPU-side scheduling and queueing remain important even when the main compute workload is accelerated by the GPU. Overall, the findings demonstrate that NVIDIA GPU acceleration can produce major performance gains, but its effectiveness depends on the interaction between algorithm design, dataset size, framework implementation, and hardware utilization.

Supplementary information. Supplementary material may include benchmark CSV files, Python source code, generated figures, Nsight Systems screenshots, GPU monitoring logs, and environment configuration files.

Acknowledgements. The author acknowledges California State University, East Bay for academic support and the instructional environment that supported this machine learning benchmarking project.

Declarations

Funding

No external funding was reported for this study.

Conflict of interest/Competing interests

The author declares no competing interests.

Ethics approval and consent to participate

Not applicable. This study used a public machine learning benchmark dataset and did not involve human participants, animal subjects, or private identifiable information.

Consent for publication

Not applicable.

Data availability

The Forest Cover Type dataset used in this study is publicly available from standard machine learning data repositories.

Code availability

The Python code used for dataset loading, GPU monitoring, metric logging, NVTX annotation, Random Forest benchmarking, XGBoost benchmarking, TensorFlow neural network training, and PyTorch neural network training is included in Appendix B.

Author contribution

William Lorenzo contributed to benchmark design, dataset preprocessing, CPU and GPU model implementation, NVIDIA GPU monitoring, Nsight Systems profiling, visualization, interpretation of results, and manuscript drafting.

Appendix A Figures

- `training_time_by_model.png`
- `speedup_by_model.png`
- `accuracy_by_model.png`
- `gpu_memory_by_model.png`
- `GPU_metrics_chart.png`
- `CPU_utilization.png`
- `thread_activity.png`

Appendix B Full Python Code

The full Python project code is maintained as separate source files in the GPU-MLBench project directory. To keep the Springer manuscript compile-stable in Overleaf, this appendix lists the exact files used in the experiment and provides representative code excerpts. The complete source files include `dataset.py`, `gpu.py`, `metrics.py`, `nvtx_utils.py`, PyTorch model and training scripts, TensorFlow model and training scripts, Random Forest CPU/GPU scripts, and XGBoost CPU/GPU scripts.

B.1 Dataset loading and preprocessing

Listing 1 Dataset loading workflow used by GPU-MLBench

```
1 def load_forest_cover_type(data_dir="datasets", rows=None,
2                             test_size=0.2, random_state=42,
3                             scale=False):
4     path = find_dataset_file(data_dir)
5     df = _read_covtype(path)
6     target_col = _target_column(df)
7     df = _sample_rows(df, target_col, rows, random_state)
8
9     y_raw = df[target_col].to_numpy()
10    target_offset = int(np.nanmin(y_raw)) if np.nanmin(y_raw) ==
11        1 else 0
12    y = y_raw.astype(np.int64) - target_offset
13
14    X_df = df.drop(columns=[target_col])
15    X_df = X_df.apply(pd.to_numeric, errors="coerce").fillna(0)
16    X = X_df.to_numpy(dtype=np.float32, copy=True)
17
18    X_train, X_test, y_train, y_test = train_test_split(
19        X, y, test_size=test_size, random_state=random_state,
20        stratify=y
21    )
22
23    if scale:
24        scaler = StandardScaler()
25        X_train = scaler.fit_transform(X_train).astype(np.float32)
26        X_test = scaler.transform(X_test).astype(np.float32)
27
28    return DatasetBundle(X_train, X_test, y_train, y_test,
29        feature_names=list(X_df.columns),
30        n_classes=int(np.max(y) + 1),
31        dataset_name="Forest Cover Type",
32        target_offset=target_offset)
```

B.2 GPU monitoring

Listing 2 NVIDIA-SMI monitoring logic

```
1 class NVidiaSMIMonitor:
2     def __enter__(self):
3         if not self.enabled or not self.available:
4             return self
5         cmd = [
6             "nvidia-smi",
7             "--query-gpu=timestamp,index,utilization.gpu,memory.
8             used",
9             "--format=csv,noheader,nounits",
10            f"-lms={self.interval_ms}",
```

```

10         ]
11         self._proc = subprocess.Popen(cmd, stdout=subprocess.PIPE
12                                     ,
13                                     stderr=subprocess.PIPE,
14                                     text=True)
15         self._reader_thread = threading.Thread(target=self.
16             _read_stdout,
17             daemon=True)
18         self._reader_thread.start()
19         return self
20
21     @staticmethod
22     def _parse_lines(lines):
23         util, mem = [], []
24         for line in lines:
25             parts = [p.strip() for p in line.split(",")]
26             if len(parts) >= 4:
27                 util.append(float(parts[-2]))
28                 mem.append(float(parts[-1]))
29         return GPUMonitorSummary(
30             gpu_memory_used_mb=max(mem) if mem else None,
31             gpu_utilization_avg_pct=sum(util) / len(util) if util

```

B.3 Benchmark result logging

Listing 3 Benchmark metric row construction

```

1 def base_result_row(model, framework, device, dataset,
2                     n_train, n_test, n_features, n_classes,
3                     training_time_sec, inference_time_sec,
4                     accuracy, library_version, params=None,
5                     data_transfer_sec=None, gpu_summary=None,
6                     notes=""):
7     row = {
8         "timestamp_utc": utc_now(),
9         "model": model,
10        "framework": framework,
11        "device": device,
12        "dataset": dataset,
13        "n_train": int(n_train),
14        "n_test": int(n_test),
15        "n_features": int(n_features),
16        "n_classes": int(n_classes),
17        "training_time_sec": float(training_time_sec),
18        "inference_time_sec": float(inference_time_sec),

```

```
19         "total_time_sec": float(training_time_sec +
20                                 inference_time_sec +
21                                 (data_transfer_sec or 0.0)),
22         "accuracy": float(accuracy),
23         "data_transfer_sec": data_transfer_sec,
24         "library_version": library_version,
25         "params_json": params_to_json(params or {}),
26         "notes": notes,
27     }
28     if gpu_summary:
29         row.update(gpu_summary)
30     return row
```

B.4 Model training scripts

The CPU and GPU training scripts followed the same structure for each model: parse benchmark arguments, load the dataset, initialize the CPU or GPU implementation, time model fitting and prediction, compute accuracy, attach GPU monitoring statistics when available, and append the result row to `benchmark_results.csv`. The project included separate scripts for `random_forest_train_cpu.py`, `random_forest_train_gpu.py`, `xgboost_train_cpu.py`, `xgboost_train_gpu.py`, `tensorflow_train_cpu.py`, `tensorflow_train_gpu.py`, `pytorch_train_cpu.py`, and `pytorch_train_gpu.py`.

Listing 4 Generic benchmark script pattern

```
1 with nvtx_range("model_fit"):
2     start = perf_counter()
3     model.fit(X_train, y_train)
4     sync_if_gpu()
5     training_time = elapsed(start)
6
7 with nvtx_range("model_predict"):
8     start = perf_counter()
9     y_pred = model.predict(X_test)
10    sync_if_gpu()
11    inference_time = elapsed(start)
12
13 acc = compute_accuracy(y_test, y_pred)
14 row = base_result_row(
15     model=model_name,
16     framework=framework_name,
17     device=device_name,
18     dataset=data.dataset_name,
19     n_train=data.n_train,
20     n_test=data.n_test,
21     n_features=data.n_features,
22     n_classes=data.n_classes,
23     training_time_sec=training_time,
```

```
24     inference_time_sec=inference_time ,  
25     accuracy=acc ,  
26     gpu_summary=monitor.summary.as_dict() if use_gpu else None ,  
27     library_version=safe_version(library_name) ,  
28     params=params ,  
29     notes=machine_notes() ,  
30 )  
31 append_result(row, results_dir)
```

References

- [1] Blackard JA, Dean DJ. Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables. *Computers and Electronics in Agriculture*. 1999.
- [2] NVIDIA Corporation. CUDA Toolkit Documentation. Available at: <https://docs.nvidia.com/cuda/>
- [3] NVIDIA Corporation. NVIDIA Nsight Systems Documentation. Available at: <https://docs.nvidia.com/nsight-systems/>
- [4] NVIDIA RAPIDS. RAPIDS cuML Documentation. Available at: <https://docs.rapids.ai/api/cuml/>
- [5] Chen T, Guestrin C. XGBoost: A scalable tree boosting system. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*; 2016.
- [6] Abadi M, Agarwal A, Barham P, et al. TensorFlow: Large-scale machine learning on heterogeneous systems. Available at: <https://www.tensorflow.org/>
- [7] Paszke A, Gross S, Massa F, et al. PyTorch: An imperative style, high-performance deep learning library. In: *Advances in Neural Information Processing Systems*; 2019.
- [8] Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, et al. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*. 2011;12:2825–2830.
- [9] McKinney W. Data structures for statistical computing in Python. In: *Proceedings of the 9th Python in Science Conference*; 2010. p. 56–61.
- [10] Harris CR, Millman KJ, van der Walt SJ, et al. Array programming with NumPy. *Nature*. 2020;585:357–362.